

CMP514 – Computer Graphics and Image Processing Laboratory

B Tech CSE TY

Activity 3 – C Graphics (graphics.h) Lab Activity: Interactive & Dynamic Drawing

Objective: Explore advanced drawing, user interaction, and more complex animation techniques.

Sub-activities:

- Master drawing complex polygons and arcs.
- Implement simple interactive elements using mouse and keyboard input.
- Animate objects with user control and more sophisticated movement.
- Understand basic concepts of screen clearing and double buffering for smoother animation.
- Explore different line styles and fill patterns.

Important Notes:

- This is a guided activity. Follow the instructions carefully.
- Continue using your graphics.h environment (DOSBox with Turbo C/C++ is recommended). Ensure your initgraph path is correct.
- Save your work often! (Usually F2 in Turbo C/C++).
- Pay attention to the comments in the code. They provide important hints.

Base Program (base_program_adv.c)

Start with a clean copy of your base program, saving it as base_program_adv.c. Further implementations will be built upon this.

```
#include <graphics.h> // Include the graphics library
#include <conio.h>      // For getch(), kbhit()
#include <stdio.h>      // For printf (optional)
#include <dos.h>         // For delay() - already available via
graphics.h usually, but good to be explicit
```

```

int main()
{
    int gd = DETECT, gm; // gd = graphics driver, gm = graphics
mode

    // Initialize graphics mode
    initgraph(&gd, &gm, "C:\\TC\\BGI"); // For Turbo C/DOSBox

    // Check if graphics initialization was successful
    if (graphresult() != grOk) {
        printf("Graphics      initialization      error:      %s\n",
grapherrormsg(graphresult()));
        getch();
        return 1;
    }

    // --- Your drawing and interaction code will go here ---

    // Example: Draw a simple line to show it's working
    // line(100, 100, 200, 200);

    // Keep the graphics window open until a key is pressed
    printf("Press any key to close the graphics window...\n"); //
This will appear in console/text mode
    getch(); // Wait for a key press

    // Close the graphics mode
    closegraph();

    return 0;
}

```

Activity Sheet

Task 0: Environment Setup and Initial Run

1. **Start your graphics.h environment:** Open DOSBox, navigate to TC\BIN, and type TC.
2. **Open the Base Program:** In Turbo C/C++, go to File > Open and navigate to C:\<YOUR_PROJECT_FOLDER>\base_program_adv.c.
3. **Compile and Run:** Press Ctrl+F9. You should see a blank graphics window (or a simple line if uncommented) that closes when you press a key.

Task 1: Drawing Complex Shapes – Polygons and Arcs

Let's draw more complex shapes.

1. Open base_program_adv.c.
2. Locate the "Your drawing and interaction code will go here" section.

Task 1.1: Draw a Polygon

A polygon is a multi-sided shape defined by an array of points.

```
// Task 1.1: Draw a simple polygon (e.g., a triangle)
setcolor(LIGHTRED);
int triangle_points[] = {
    100, 100, // Top point (x1, y1)
    200, 50,  // Right point (x2, y2)
    300, 100, // Bottom point (x3, y3)
    100, 100  // Close the polygon by repeating the first point
};
// drawpoly(number_of_points, array_of_points)
// The number of points here is 4 (x1,y1, x2,y2, x3,y3, x1,y1)
// total 8 values, so 4 points.
drawpoly(4, triangle_points);

// Experiment: Try drawing a square or a house shape using
// drawpoly.
// Remember to repeat the first point at the end to close the
// shape.
```

Save, Compile, and Run (Ctrl+F9). Observe the drawn polygon.

Task 1.2: Draw an Arc

An arc is a segment of a circle.

```
// Task 1.2: Draw an arc
setcolor(LIGHTGREEN);
// arc(centerX, centerY, startAngle, endAngle, radius)
// Angles are in degrees, 0 is right, 90 is down, 180 is left, 270
is up.
arc(400, 150, 0, 180, 80); // Draws a semi-circle opening downwards

// Experiment: Change the startAngle and endAngle to draw different
parts of a circle.
// Try drawing a "Pac-Man" shape.
```

Save, Compile, and Run. Observe the arc.

Task 1.3: Filled Polygon

You can also draw and fill polygons directly.

```
// Task 1.3: Draw and fill a different polygon
setcolor(LIGHTMAGENTA);
setfillstyle(HATCH_FILL, YELLOW); // Try a different fill style,
e.g., HATCH_FILL, WIDE_DOT_FILL
int square_points[] = {
    50, 250,
    150, 250,
    150, 350,
    50, 350,
    50, 250
};
fillpoly(5, square_points); // fillpoly(number_of_points,
array_of_points)
```

```
// Experiment: Try different setfillstyle options. (e.g.,  
EMPTY_FILL, LINE_FILL, LTBK_FILL)
```

```
// Also experiment with different fill colors.
```

Save, Compile, and Run. Observe the filled polygon.

Task 2: User Interaction – Mouse and Keyboard Input

This is the way to make your graphics ‘dynamic’!

Task 2.1: Wait for a specific key press

Instead of just `getch()`, let's make the program react to a specific key.

```
// Task 2.1: Wait for 'Q' key press to quit  
setcolor(WHITE);  
outtextxy(10, 10, "Press 'Q' to quit this section.");  
  
// Instead of getch() at the end of main, you can use a loop  
// for this section's interaction.  
char ch;  
do {  
    ch = getch(); // Read a character  
} while (ch != 'q' && ch != 'Q'); // Loop until 'q' or 'Q' is  
pressed  
  
// You can clear the screen after this section if you want  
// cleardevice();
```

Integrate this loop after your drawing code from Task 1, and before the final `getch()` and `closegraph()`. Save, Compile, and Run. The program should now wait for 'Q' before proceeding.

Task 2.2: Simple Mouse Click Detection

`graphics.h` has limited direct mouse support for complex events, but we can detect if a mouse button is pressed and get its coordinates.

```

// Task 2.2: Detect mouse clicks and draw a circle at click
location
cleardevice(); // Clear screen for this task
setcolor(WHITE);
outtextxy(10, 10, "Click anywhere to draw a circle. Press 'Q' to
quit.");

int mouse_x, mouse_y;
char key_pressed;

// Loop until 'Q' is pressed
while (!kbhit() || (key_pressed = getch()) != 'q' && key_pressed
!= 'Q') {
    if (ismousebuttondown(WM_LBUTTONDOWN)) { // Check if left
mouse button is down
        mouse_x = mousex(); // Get current mouse X coordinate
        mouse_y = mousey(); // Get current mouse Y coordinate

        setcolor(CYAN);
        circle(mouse_x, mouse_y, 20); // Draw a circle at the
clicked location
        delay(100); // Small delay to prevent multiple circles
from one click
    }
    delay(10); // Small delay to prevent CPU hogging in the loop
}
// After the loop, you might want to consume the 'Q' character if
it ended the loop
if (key_pressed == 'q' || key_pressed == 'Q') {
    // No action needed, the loop already consumed it.
}
cleardevice(); // Clear for next task

```

Integrate this after Task 2.1's code. Remember to include dos.h or ensure delay() is available. Save, Compile, and Run. Click around the screen and see circles appear.

Task 2.3: Keyboard Controlled Movement

Let's make an object move based on arrow keys.

```
// Task 2.3: Keyboard controlled square
cleardevice(); // Clear screen for this task
setcolor(WHITE);
outtextxy(10, 10, "Use Arrow Keys to move the square. Press 'Q' to
quit.");

int sq_x = getmaxx() / 2 - 25; // Start in middle
int sq_y = getmaxy() / 2 - 25;
int sq_size = 50;
char key;

// Main animation/interaction loop
while (1) {
    // Draw the square
    setcolor(YELLOW);
    rectangle(sq_x, sq_y, sq_x + sq_size, sq_y + sq_size);

    // Check for key press
    if (kbhit()) { // If a key is pressed
        key = getch(); // Get the key

        // Special handling for arrow keys (which are two-byte
sequences)
        if (key == 0) { // Indicates a special key (like arrow
keys)
            key = getch(); // Read the second byte

            // Define keys for arrow keys (these are common BIOS
scan codes)
            // Up: 72, Down: 80, Left: 75, Right: 77
```

```

        switch (key) {
            case 72: sq_y -= 5; break; // Up arrow
            case 80: sq_y += 5; break; // Down arrow
            case 75: sq_x -= 5; break; // Left arrow
            case 77: sq_x += 5; break; // Right arrow
        }
    } else if (key == 'q' || key == 'Q') {
        break; // Exit loop if 'Q' is pressed
    }
}

delay(20); // Small delay to control speed

// "Erase" the old square by redrawing it in black
setcolor(BLACK);
rectangle(sq_x, sq_y, sq_x + sq_size, sq_y + sq_size);

// Keep square within bounds (optional but good practice)
if (sq_x < 0) sq_x = 0;
if (sq_x > getmaxx() - sq_size) sq_x = getmaxx() - sq_size;
if (sq_y < 0) sq_y = 0;
if (sq_y > getmaxy() - sq_size) sq_y = getmaxy() - sq_size;
}
cleardevice(); // Clear for next task

```

Integrate this after Task 2.2's code. Save, Compile, and Run. Use arrow keys to move the square. Press 'Q' to quit.

Task 3: Advanced Animation Concepts

Let's make animation smoother and more dynamic.

Task 3.1: Introducing `setactivepage()` and `setvisualpage()` (Double Buffering)

The "erase and redraw" method can cause flickering. Double buffering uses two memory pages: you draw on a "hidden" page (`setactivepage()`), then quickly display it (`setvisualpage()`).

```

// Task 3.1: Smooth Animation using Double Buffering
cleardevice(); // Clear screen
setcolor(WHITE);
outtextxy(10, 10, "Smooth moving circle (double buffered). Press
'Q' to quit.");

int circle_x = 50;
int circle_y = getmaxy() / 2;
int radius = 30;
int direction = 1; // 1 for right, -1 for left
int page = 0; // Current active page

// Loop for animation
while (1) {
    setactivepage(page); // Set drawing to the hidden page
    cleardevice();       // Clear the hidden page

    // Draw the circle on the hidden page
    setcolor(LIGHTBLUE);
    circle(circle_x, circle_y, radius);

    // Display the hidden page
    setvisualpage(page);

    // Toggle pages for next frame
    page = 1 - page; // Switches between 0 and 1

    // Update position
    circle_x += (5 * direction);

    // Boundary check and change direction
    if (circle_x + radius > getmaxx() || circle_x - radius < 0) {
        direction *= -1; // Reverse direction
    }
}

```

```

    }

    delay(20); // Control animation speed

    // Check for 'Q' to quit
    if (kbhit()) {
        char k = getch();
        if (k == 'q' || k == 'Q') {
            break;
        }
    }
}

// Before exiting, ensure you are on the default page (page 0)
setactivepage(0);
setvisualpage(0);
cleardevice();

```

Integrate this after Task 2.3's code. Save, Compile, and Run. Observe the significantly smoother animation compared to the previous single-buffered example.

Task 3.2: Simple Ball Bounce (Physics-like Movement)

Combine movement with boundary detection for a bouncing effect.

```

// Task 3.2: Bouncing Ball
cleardevice();
setcolor(WHITE);
outtextxy(10, 10, "Bouncing Ball. Press 'Q' to quit.");

int ball_x = getmaxx() / 2;
int ball_y = getmaxy() / 2;
int ball_dx = 5; // X velocity
int ball_dy = 3; // Y velocity
int ball_radius = 15;

```

```

page = 0; // Reset page for this animation

while (1) {
    setactivepage(page);
    cleardevice();

    setcolor(WHITE);
    setfillstyle(SOLID_FILL, RED);
    circle(ball_x, ball_y, ball_radius);
    floodfill(ball_x, ball_y, WHITE); // Fill the circle

    setvisualpage(page);
    page = 1 - page;

    // Update position
    ball_x += ball_dx;
    ball_y += ball_dy;

    // Check for horizontal boundaries
    if (ball_x + ball_radius > getmaxx() || ball_x - ball_radius
< 0) {
        ball_dx *= -1; // Reverse X direction
    }
    // Check for vertical boundaries
    if (ball_y + ball_radius > getmaxy() || ball_y - ball_radius
< 0) {
        ball_dy *= -1; // Reverse Y direction
    }

    delay(20);

    if (kbhit()) {
        char k = getch();
    }
}

```

```

        if (k == 'q' || k == 'Q') {
            break;
        }
    }
}
setactivepage(0);
setvisualpage(0);
cleardevice();

```

Integrate this after Task 3.1's code. Save, Compile, and Run. Watch the ball bounce around the screen.

Task 3.3: Drawing with Different Line Styles

Beyond just color, you can change how lines are drawn.

```

// Task 3.3: Line Styles
cleardevice();
setcolor(WHITE);
outtextxy(10, 10, "Different Line Styles. Press any key to
close.");

// setlinestyle(linestyle, pattern, thickness)
// Line styles: SOLID_LINE, DOTTED_LINE, CENTER_LINE, DASHED_LINE,
USERBIT_LINE
// Thickness: NORM_WIDTH (1 pixel), THICK_WIDTH (3 pixels)

// Solid Line, Normal Width
setlinestyle(SOLID_LINE, 0, NORM_WIDTH);
setcolor(RED);
line(50, 50, 200, 50);
outtextxy(50, 60, "Solid Line");

// Dotted Line, Normal Width
setlinestyle(DOTTED_LINE, 0, NORM_WIDTH);

```

```

setcolor(GREEN);
line(50, 100, 200, 100);
outtextxy(50, 110, "Dotted Line");

// Dashed Line, Thick Width
setlinestyle(DASHED_LINE, 0, THICK_WIDTH);
setcolor(BLUE);
line(50, 150, 200, 150);
outtextxy(50, 160, "Dashed Line (Thick)");

// Center Line, Normal Width
setlinestyle(CENTER_LINE, 0, NORM_WIDTH);
setcolor(YELLOW);
line(50, 200, 200, 200);
outtextxy(50, 210, "Center Line");

// Reset to default for subsequent drawings if needed
setlinestyle(SOLID_LINE, 0, NORM_WIDTH);

```

Integrate this after Task 3.2's code, but before the final getch(). Save, Compile, and Run. Observe the different line appearances.

Attempt this:

- How does **double buffering** improve animation quality compared to single-page drawing?
- Describe a scenario where **mouse input** would be essential in a graphics application.
- How would you create a simple **drawing program** where a user could "draw" on the screen by dragging the mouse? List the functions you would use.

Troubleshooting Tips:

- **"My mouse clicks aren't detected!":** Ensure you're linking correctly for mouse functions. Sometimes, specific DOSBox configurations or older Turbo C++ versions might have limited `mousex()`, `mousey()`, `ismousebuttondown()` support. Test with a simpler `kbhit()` first.
- **"My arrow keys don't work!":** Arrow keys (and other special keys like F-keys) often return 0 as the first byte, then a second byte for the actual key code. Ensure your `kbhit()` and `getch()` logic handles this (`if (key == 0) { key = getch(); ... }`).
- **"My double buffering just shows a blank screen or flickers still!":**
 - Ensure `setactivepage()` and `setvisualpage()` are correctly toggling (`page = 1 - page;`).
 - Make sure you are calling `cleardevice()` *after* `setactivepage()` and *before* drawing on that active page.
 - The `delay()` value might be too low or too high; experiment with it.
- **General graphics.h issues:** Always re-verify your `initgraph` path and ensure BGI files are present.

Design following 3 drawings using the functions of `graphics.h` and consider adding animation effects best to your imagination.



